ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E. Information Technology

V-SEMESTER

22ITCP508 – SYSTEM SOFTWARE AND COMPILER DESIGN

LABORATORY RECORD

(July 2026 – December 2026)

Name : _____

Register No. : _____

ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E. Information

Technology V-

SEMESTER

22ITCP508 – SYSTEM SOFTWARE AND COMPILER DESIGN LAB

Certified that this is the bona-fide record of work done by Mr./Ms._____

Reg. No._____of B.E. (Information Technology) in the

22ITCP508 – SYSTEM SOFTWARE AND COMPILER DESIGN

Laboratory during the v-th semester of the academic year 2026 – 2027.

Place: Annamalai Nagar
Date: / / 2026. **Staff in-charge**

Internal Examiner **External Examiner**

DEPARTMENT OF INFORMATION TECHNOLOGY

B.E INFORMATION TECHNOLOGY

VISION:

To produce globally competent, quality technocrats, to inculcate values of leadership and research qualities and to play a vital role in the socio-economic progress of the nation.

MISSION:

1. To partner with the University community to understand the information technology needs of faculty, staff and students.
2. To develop dynamic IT professionals with globally competitive learning experience by providing high class education.
3. To involve graduates in understanding need-based Research activities and disseminate the knowledge to develop entrepreneur skills.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO	PEO Statements
PEO1	To offer students with **core competence** in mathematical, scientific and basic engineering rudiments necessary to prepare, analyze and solve hardware/software engineering problems and/or also to pursue advanced study or research.
PEO2	To educate students with good **scope** of knowledge in core areas of IT and related engineering so as to comprehend engineering trade-offs, analyze, design, and synthesize data and technical concepts to create novel products and solutions for the real-life problems.
PEO3	To instil in students to maintain high **proficiency** and ethical standards, effective oral and written communication skills, to work as part of teams on multidisciplinary projects and diverse professional environments, and relate engineering issues to the society, globa economy and to emerging technologies.
PEO4	To deliver our graduates with **learning environment** awareness of the life-long learning needed for a successful professional career and to introduce them to written ethical codes and guidelines, perform excellence, leadership and demonstrate good citizenship.

22ITCP508	SYSTEM SOFTWARE AND COMPILER DESIGN LAB	L	T	P	C
		0	0	3	1.5

COURSE OBJECTIVES

- Be familiar with the main features of C language
- Understands Assembler and loader paradigm
- Apply the concepts of parser

LIST OF EXERCISES (SYSTEM SOFTWARE)

1. Implementation of Token Separation.
2. Implementation of Symbol table Manipulation
3. Implementation of pass one of Two pass Assembler
4. Implementation of pass Two of Two pass Assembler
5. Implementation of one pass Assembler
6. Simulation of absolute loader
7. Simulation of Linking Loader
8. Implementation of Inter-process Communication
9. Implementation of Macro Substitution
10. Design of Editor

LIST OF EXERCISES (COMPILER DESIGN)

1. Design a lexical analyzer for given language and the lexical analyzer should ignore redundant spaces, tabs and new lines. It should also ignore comments. Although the syntax specification states that identifiers can be arbitrarily long, you may restrict the length to some reasonable value. Simulate the same in C language.
2. Write a C program to identify whether a given line is a comment or not
3. Write a C program to recognize strings under 'a', 'a*b+', 'abb'
4. Write a C program to test whether a given identifier is valid or not.
5. Write a C program to simulate lexical analyzer for validating operators
6. Implement the lexical analyzer using JLex, flex or other lexical analyzer generating tools.

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs)															
	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**	**PSO3**
CO1	3	2	3	3	3	-	-	-	-	-	-	-	3	2	2
CO2	3	3	3	3	2	-	-	-	2	-	-	-	3	2	1
CO3	3	3	3	3	2	-	-	-	1	-	-	-	3	2	1

Rubric for CO3

Rubric for CO3 in Laboratory Courses					
Rubric	**Distribution of 10 Marks for CIE/SEE Evaluation Out of 40/60 Marks**				
	Up To 2.5 Marks	**Up To 5 Marks**	**Up To 7.5 Marks**	**Up To 10 marks**	**Up To 2.5 Marks**
Demonstrate an ability to listen and answer the viva questions related to programming skills needed for solving real-world problems in Computer Science and Engineering.	Poor listening and communication skills. Failed to relate the programming skills needed for solving the problem.	Showed better communication skill by relating the problem with the programming skills acquired but the description showed serious errors.	Demonstrated good communication skills by relating the problem with the programming skills acquired with few errors.	Demonstrated excellent communication skills by relating the problem with the programming skills acquired and have been successful in tailoring the description.	Demonstrate an ability to listen and answer the viva questions related to programming skills needed for solving real-world problems in Computer Science and Engineering.

List of Exercises

S. No.	Date	Experiment Name	Page No.	Marks	Signature
1.		Implementation of lexical for IF statement			
2.		Implementation of lexical analyzer for arithmetic expression			
3.		Implementation of lexical analyzer using LEXTOOL			
4.		Construction of NFA from regular expression			
5.		Construction of DFA from NFA			
6.		Implementation of shift reduce parsing algorithm			
7.		Implementation of operator precedence parser			
8.		Implementation of recursive descent parser			
9.		Implementation of code optimization techniques			
10.		Implementation of code generator			
11.		Implementation of symbol table			

Exp No: 1	Implementation of Lexical Analyzer for 'if' Statement
Date:	

AIM:

To design and implement a Lexical Analyzer in Python that identifies and classifies tokens such as identifiers, numbers, operators, assignment symbols, and delimiters from a given input string.

ALGORITHM:

1. Start the program.
2. Read the input source code line that contains the „if" statement, for example if a > b:.
3. Initialize an empty list to store tokens and define sets of keywords, operators, and symbols
4. to identify different token types.
5. Scan the source code character by character and separate it into meaningful lexemes such as keywords, identifiers, operators, and symbols.
6. Compare each lexeme with the predefined sets to determine its type; for example, classify
7. „if" as a keyword, „a" and „b" as identifiers, and „>" as an operator.
8. Store each recognized lexeme along with its corresponding token type in the token list.
9. Display all the tokens with their types as output and stop the program.

PROGRAM:

```
import re

vars = []
vcnt = 0
input_str = input("Enter the Input String: ") token = ""
state = 0
pos = 0
tlen = 0

def getAddress(str_):
    global vcnt
    for i in range(vcnt):
        if vars[i] == str_:
            return v
```

```
vars.append(str_) vcnt += 1
    return vars[vcnt - 1]

def isrelop(c):
    return c in ['+', '-', '*', '/', '%', '^']

while pos < len(input_str):
    c = input_str[pos]

    if state == 0:
        if c.isspace():
            pass  # Ignore whitespace

        elif c.isalpha():
            token = c
            tlen = 1
            state = 1

        elif c.isdigit():
            token = c
            tlen = 1
            state = 2

        elif isrelop(c):
            token = c
            tlen = 1
            state = 3

        elif c == ';':
            print(f"{c}\t<3,3>")

        elif c == '=':
            print(f"{c}\t<4,4>")

    elif state == 1:
        # Identifier
        if c.isalnum():
            token += c
            tlen += 1
        else:
            print(f"{token}\t<1,{getAddress(token)}>")
```

```
                state = 0
            pos -= 1  # Reprocess this character in next loop

        elif state == 2:
            # Number
            if c.isdigit():
                token += c
                tlen += 1
            else:
                print(f"{token}\t<2,{token}>")
                state = 0
                pos -= 1

        elif state == 3:
            # Operator
            if c == '=':
                print(f"{token + c}\t<{ord(token)*10},{ord(token)*10}>")
            else:
                print(f"{token}\t<{ord(token)},{ord(token)}>")

            pos -= 1
            state = 0

        pos += 1

    # Final token check in case input ends with identifier or number
    if state == 1:
        print(f"{token}\t<1,{getAddress(token)}>")

    elif state == 2:
        print(f"{token}\t<2,{token}>")

    elif state == 3:
        print(f"{token}\t<{ord(token)},{ord(token)}>")
```

SAMPLE INPUT & OUTPUT:

Enter the Input String: if(a>=b) max=a;

if	<1,if>
a	<1,a>
=	<4,4>
b	<1,b>
max	<1,max>
=	<4,4>
a	<1,a>
;	<3,3>

RESULT :

The lexical analyzer was implemented successfully. Tokens for keywords, identifiers, and operators were recognized correctly.

Exp No: 2	Implementation of Lexical Analyzer for arithmetic operation
Date:	

AIM:

To implement a lexical analyzer in Python that identifies and classifies tokens such as identifiers, operators, numbers, and symbols from an arithmetic expression.

ALGORITHM:

1. Start the program.
2. Read the input expression.
   a. Accept an arithmetic expression as input from the user (for example, a + b * c - 5).
   b. Store it as a string for further processing.
3. Initialize data structures.
   a. Create an empty list named tokens to store the identified tokens.
   b. Define sets or lists for operators (+, -, *, /, %), parentheses ((,)), and digits or identifiers.
4. Scan the input expression.
   a. Read the input character by character.
   b. Group alphabetic or numeric sequences into valid lexemes (identifiers or numbers).
   c. Recognize individual symbols such as +, -, *, /, and classify them as operators.
5. Classify each lexeme.
   a. If a lexeme is made of alphabets, classify it as an Identifier.
   b. If a lexeme is made of digits, classify it as a Number.
   c. If a character is in the operator list, classify it as an Operator.
   d. If a character is a parenthesis, classify it as a Symbol.
6. Store tokens.
   a. For each identified lexeme, create a pair like (token_type, lexeme)and add it to the tokens. list
7. Display the tokens.
   a. Print all tokens along with their types in a structured format.
   b. This helps visualize how the lexical analyzer separates components of the arithmetic expression.
8. Stop the program.

`

PROGRAM:

```
vars = []
vcnt = 0
state = 0
pos = 0
token = ""
tlen = 0

def getAddress(str_):
    global vcnt
    for i in range(vcnt):
        if vars[i] == str_:
            return i
    vars.append(str_)
    vcnt += 1
    return vcnt - 1

def isrelop(c):
    return c in ['+', '-', '*', '/', '%', '^']

input_str = input("Enter the Input String: ")

while pos < len(input_str):
    c = input_str[pos]
    print(c, end='')

    if state == 0:
        if c.isspace():
            pass  # Ignore whitespace

        elif c.isalpha():
            token = c
            tlen = 1
            state = 1

        elif c.isdigit():
            token = c
            tlen = 1
            state = 2

        elif isrelop(c):
            print(f"\t<{ord(c)},{ord(c)}>")

        elif c == ';':
            print("\t<3,3>")

        elif c == '=':
            print("\t<4,4>")
    elif state == 1:
```


`

```
            # Identifier
            if not c.isalnum():
                print(f"\t<1,{getAddress(token)}>")
                state = 0
                pos -= 1
            else:
                token += c
                tlen += 1

        elif state == 2:
            # Number
            if not c.isdigit():
                print(f"\t<2,{token}>")
                state = 0
                pos -= 1
            else:
                token += c
                tlen += 1

        pos += 1

    # Flush last token if needed
    if state == 1:
        print(f"\t<1,{getAddress(token)}>")

    elif state == 2:
        print(f"\t<2,{token}>")
```

SAMPLE INPUT & OUTPUT

Enter the Input String: a=a*2+b/c; a=

a=	<1,0>
=	<4,4>
a*	<1,0>
*	<42,42>
2+	<2,2>
+	<43,43>
b/	<1,1>
/	<47,47>
c;	<1,2>
;	<3,3>

RESULT :

The program executed successfully, Arithmetic expressions were tokenized correctly with proper operator recogniztion.

`

Exp No: 3	Implementation of Lexical Analyzer using LEXTOOL
Date:	

AIM:

To implement a lexical analyzer using the LEXTOOL that recognizes and classifies tokens such as identifiers, operators, numbers, and symbols from an arithmetic expression.

ALGORITHM:

1. Start the program.
2. Define the token patterns for identifiers, keywords, operators, numbers, and symbols using regular expressions in the LEX specification file (`.l` file).
3. Include necessary header files (e.g., `stdio.h`) in the declaration section of the LEX program.
4. Write the rules section, where each token pattern is associated with an action (e.g., printing the type of token recognized).
5. Add the main() function to initialize the lexical analysis process and call `yylex()` to begin scanning the input.
6. Compile and execute the LEX program using the following commands:
    - `lex filename.l` → generates `lex.yy.c`
    - `gcc lex.yy.c -o output` → compiles the generated C file
    - `./output` → runs the lexical analyzer
7. Stop the program.

PROGRAM:

```
import re

# Keywords and operators
keywords = [
    "int", "void", "main", "char", "if", "for", "while", "else", "printf", "scanf",
    "FILE", "include", "stdio.h", "conio.h", "iostream.h"]
`
```

```
    operators = {
    "(": "openpara", ")": "closepara", "{": "openbrace", "}": "closebrace",
    "<": "lesser", ">": "greater", "": "doublequote", "'": "singlequote",
    ":": "colon", ";": "semicolon", "#": "preprocessor", "=": "equal",
    "==": "assign", "%": "percentage", "^": "bitwise", "&": "reference",
    "<": "lesser", ">": "greater", "": "doublequote", "'": "singlequote",

    ":": "colon", ";": "semicolon", "#": "preprocessor", "=": "equal",
    "==": "assign", "%": "percentage", "^": "bitwise", "&": "reference",
    "*": "star", "+": "add", "-": "sub", "\\": "backslash", "/": "slash"
}

# Sample input code.
input_code = """ #include "stdio.h" #include
 {
“ ” ”
int a=10; getch();
}"""

# Tokenization
tokens = []
current_token = ""

for c in input_code:
    if c.isalnum() or c in ['[', ']', '.']:
        current_token += c
    else:
        if current_token:
            tokens.append(current_token)
            current_token = ""

        if c == '\n':
            tokens.append('$')  # Line marker
        elif not c.isspace():
            tokens.append(c)
# Append last token if present
if current_token:
    tokens.append(current_token)
# Lexical analysis
print("Lexical Analysis")
line = 1
print(f"\nLine: {line}")
for token in tokens:
    if token == '$':  # Line break marker
        line += 1
        print(f"\nLine: {line}")

`
```

```
        continue
if token in operators:
      print(f"\t\t{token}\t:\t{operators[token]}")
elif token in keywords:
      print(f"\t\t{token}\t:\tKeyword")
elif token.isdigit():
       print(f"\t\t{token}\t:\tConstant")
 else:
       print(f"\t\t{token}\t:\tIdentifier")
```

SAMPLE INPUT/OUTPUT:

Lexical Analysis

```
 Line: 1
        #       :       preprocessor
        include :       Keyword
        "       :       doublequote
        stdio.h :       Keyword
        "       :       doublequote
        #       :       preprocessor
       include :        Keyword

 Line: 2
      {       :       openbrace

 Line: 3
        int     :       Keyword
        a       :       Identifier
        =       :       equal
        10      :       Constant
        ;       :       semicolon
       getch    :       Identifier
       (        :       openpara
       )        :       closepara
       ;        :       semicolon

Line: 4
       }        :       closebrace

Line: 5
```

RESULT :

The lexical analyzer using LEXTOOL worked as expected. Tokens were generated accurately for given inputs.

`

Exp No: 4	Construction of NFA from Regular Expression
Date:	

AIM:

To construct a Non-deterministic Finite Automata (NFA) from a given Regular Expression using Thompson‟s Construction method.

ALGORITHM:

1) Start the program.
2) Read the regular expression as input.
3) Scan the regular expression from left to right and identify the operators:
   a) Concatenation (·)
   b) Union (|)
   c) Kleene Star (*)
4) Apply Thompson‟s Construction Rules based on the operator:
   a) For a single symbol (a):
      Create two states with a transition labeled „a‟.
   b) For concatenation (AB):
      Connect the final state of NFA for A to the start state of NFA for B.
   c) For union (A | B):
      Create a new start and new final state; connect the new start state to the start states of A and B using ε-transitions, and their final states to the new final state using ε-transitions.
   d) *For Kleene star (A):**
      Create a new start and new final state; connect the new start state to A‟s start state and to the new final state using ε-transitions; also connect A‟s final state to its start state and to the new final state using ε-transitions.
5) Repeat the above steps until the entire regular expression is converted into an equivalent NFA.
6) Display the constructed NFA showing all states and transitions.
7) Stop the program**.**

`

`

PROGRAM:

```
import networkx as nx
import matplotlib.pyplot as plt

# Global counter for unique state names
state_counter = 0

def new_state():
    global state_counter
    state = "S" + str(state_counter)
    state_counter += 1
    return state

# NFA class
class NFA:
    def __init__(self, start, end):
        self.start = start
        self.end = end
        self.transitions = {}

    def add_transition(self, from_state, symbol, to_state):
        if from_state not in self.transitions:
            self.transitions[from_state] = {}

        if symbol not in self.transitions[from_state]:
            self.transitions[from_state][symbol] = []

        self.transitions[from_state][symbol].append(to_state)

    def merge(self, other):
        for state in other.transitions:
            if state not in self.transitions:
                self.transitions[state] = {}

            for symbol in other.transitions[state]:
                if symbol not in self.transitions[state]:
                    self.transitions[state][symbol] = []
```

`

```
            self.transitions[state][symbol] += other.transitions[state][symbol]

# Create NFA for a single character
def char_nfa(symbol):
    start = new_state()
    end = new_state()

    nfa = NFA(start, end)
    nfa.add_transition(start, symbol, end)

    return nfa

# Kleene star (*)
def star_nfa(nfa):
    start = new_state()
    end = new_state()

    result = NFA(start, end)
    result.add_transition(start, 'e', nfa.start)
    result.add_transition(start, 'e', end)
    result.add_transition(nfa.end, 'e', nfa.start)
    result.add_transition(nfa.end, 'e', end)

    result.merge(nfa)

    return result

# Concatenation
def concat_nfa(nfa1, nfa2):
    result = NFA(nfa1.start, nfa2.end)
    result.merge(nfa1)
    result.merge(nfa2)

    result.add_transition(nfa1.end, 'e', nfa2.start)

    return result
```

`

```
# Union (using '/')
def union_nfa(nfa1, nfa2):
    start = new_state()
    end = new_state()

    result = NFA(start, end)
    result.add_transition(start, 'e', nfa1.start)
    result.add_transition(start, 'e', nfa2.start)
    result.add_transition(nfa1.end, 'e', end)
    result.add_transition(nfa2.end, 'e', end)

    result.merge(nfa1)
    result.merge(nfa2)

    return result


# Convert regex to NFA
def regex_to_nfa(regex):
    stack = []
    i = 0

    while i < len(regex):
        c = regex[i]

        if c.isalpha():
            nfa = char_nfa(c)

            if i + 1 < len(regex) and regex[i + 1] == '*':
                nfa = star_nfa(nfa)
                i += 1

            stack.append(nfa)

        elif c == '(':
            j = i
            i += 1

            balance = 0

            while i < len(regex):
`
```

```
            if regex[i] == '(':
                balance += 1
            elif regex[i] == ')':
                balance -= 1

                if balance == 0:
                    break

            i += 1

        sub_nfa = regex_to_nfa(regex[j + 1:i])

        if i + 1 < len(regex) and regex[i + 1] == '*':
            sub_nfa = star_nfa(sub_nfa)
            i += 1

        stack.append(sub_nfa)

    elif c == '/':
        nfa2 = stack.pop()
        i += 1
        nfa3 = regex_to_nfa(regex[i:])
        return union_nfa(nfa2, nfa3)

    i += 1

result = stack[0]

for nfa in stack[1:]:
    result = concat_nfa(result, nfa)

return result

# Draw the NFA graph
    def draw_nfa(nfa):
        G = nx.MultiDiGraph()

        for from_state in nfa.transitions:
            for symbol in nfa.transitions[from_state]:
                for to_state in nfa.transitions[from_state][symbol]:
```

`

```
    G.add_edge(from_state, to_state, label=symbol)

    pos = nx.spring_layout(G)

    nx.draw(
        G,
        pos,
        with_labels=True,
        node_size=2000,
        node_color="lightyellow",
        font_size=10
    )

    edge_labels = {
        (u, v): d['label']
        for u, v, d in G.edges(data=True)
    }

    nx.draw_networkx_edge_labels(
        G,
        pos,
        edge_labels=edge_labels
    )

    plt.title("NFA from Regular Expression")
    plt.show()

# MAIN block
if __name__ == "__main__":
    print("Enter a regular expression using: characters, *, /, ( )")
    print("Note: Union → / | Kleene Star → * | Epsilon transitions are shown as 'e'")
    print("Example: a(b/c)*d")

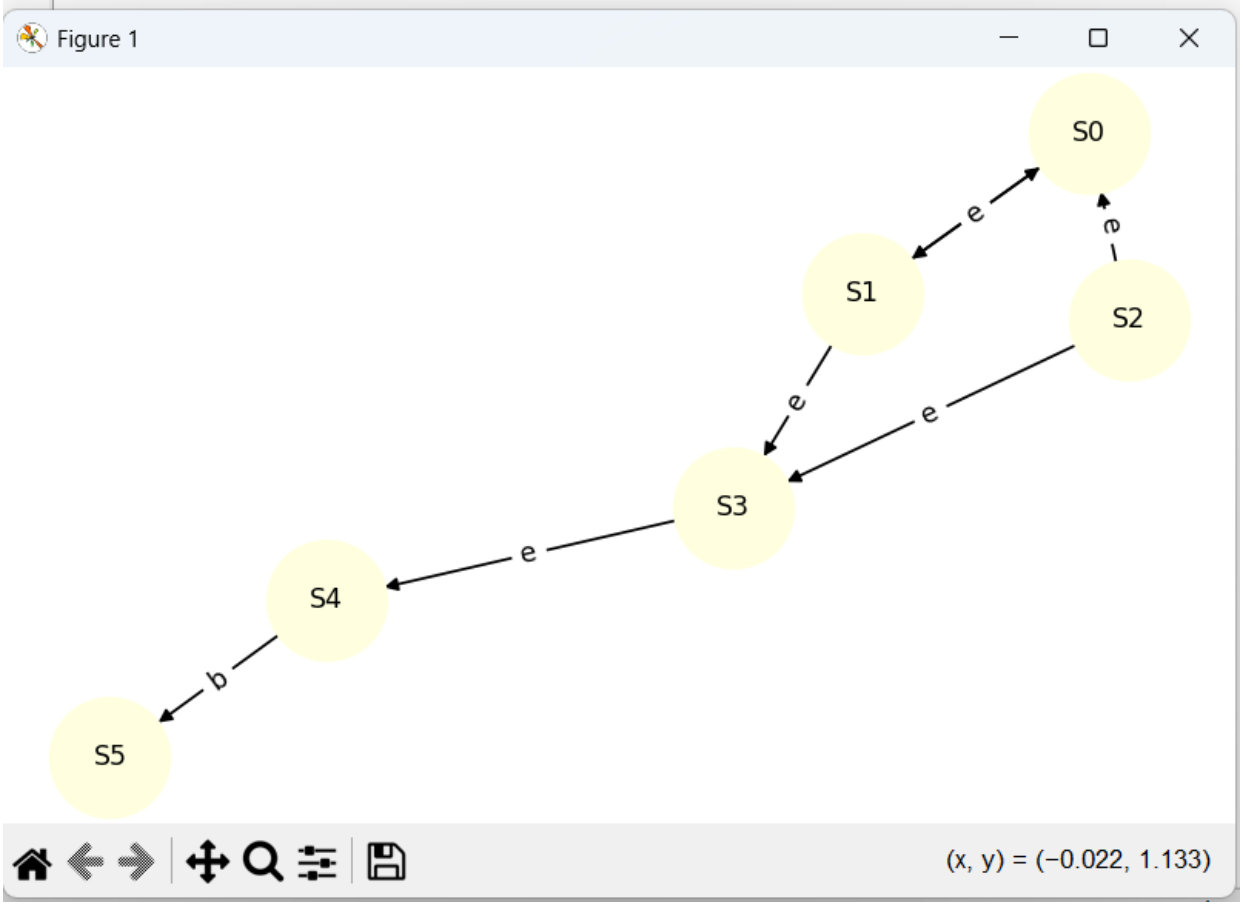
    regex = input("Enter Regular Expression: ")
    nfa = regex_to_nfa(regex)

    draw_nfa(nfa)
```

`

Sample inputs/outputs:

```
====================== RESTART: C:\code\lab\syssoft\e4.py ==================
Enter a regular expression using: characters, *, /, ( )
Note: Union → /  | Kleene Star → *  | Epsilon transitions are shown as 'e'
Example: a(b/c)*d
Enter Regular Expression: a*b
```



RESULT :

The NFA was constructed successfully using Thompson‟s method. All transitions and states were verified

Exp No: 5	Construction of DFA from NFA
Date:	

AIM:

To construct a Deterministic Finite Automaton (DFA) equivalent to a given Non-Deterministic Finite Automaton (NFA) using the Subset Construction Method.

ALGORITHM:

1) Start the program.
2) Read the given NFA, including:
   a) The set of states (Q)
   b) The input symbols (Σ)
   c) The transition function (δ)
   d) The start state (q_0)
   e) The final states (F)
3) Compute the ε-closure of the NFA start state to form the start state of the DFA.
4) For each DFA state, perform the following:
   a. For each input symbol, find the set of NFA states reachable (including ε-moves).
   b. If this new set of states has not been added as a DFA state, add it to the DFA‟s state list.
   c. Record the transition from the current DFA state to the new DFA state for the given input symbol.
5) Repeat Step 4 until no new DFA states are produced.
6) Mark all DFA states as final if they contain at least one final state of the NFA.
7) Stop the program.

`

PROGRAM:

```
from collections import defaultdict, deque

# Step 1: Input NFA
n = int(input("Enter number of NFA states: "))

print("Enter the NFA transition table row by row.")
print("Use 'e' for epsilon, 'a'/'b' for symbols, '.' for none.")
print("Example for 3 states: e.a ..a ...")

# Initialize transition table
data = [[" for _ in range(n + 1)] for _ in range(n + 1)]

for i in range(1, n + 1):
    row = input(f"State {i}: ").replace(" ", "")

    if len(row) != n:
        raise ValueError("Invalid row length. Must be exactly n characters.")

    for j in range(1, n + 1):
        data[i][j] = row[j - 1]

# Step 2: Compute epsilon closures
e_close = defaultdict(list)


def compute_epsilon_closure(state):
    closure = set()
    stack = [state]

    while stack:
        s = stack.pop()
        if s not in closure:
            closure.add(s)
            for j in range(1, n + 1):
                if data[s][j] == 'e':  # epsilon transition
                    stack.append(j)
    return sorted(closure)
# Calculate epsilon closure for each state
for i in range(1, n + 1):
    e_close[i] = compute_epsilon_closure(i)
# Display epsilon closures
print("\nEpsilon Closures:")
for i in range(1, n + 1):
    print(f"E({i}) = {{{', '.join(map(str, e_close[i]))}}}")
```

`

```
    # Step 3: Subset Construction for DFA
symbol_set = ['a', 'b']  # Assuming alphabet is {a, b}
dfa_states = []
dfa_transitions = {}
def move(states, symbol):
    result = set()
    for state in states:
        for j in range(1, n + 1):
            if data[state][j] == symbol:
                result.update(e_close[j])
    return sorted(result)
# Initial DFA state = epsilon closure of NFA start state
initial = e_close[1]
dfa_states.append(initial)
queue = deque([initial])
print("\nDFA Transition Table:")
print(f"{'State':<15}{'a':<20}{'b':<20}")
print("-" * 55)
while queue:
    current = queue.popleft()
    current_label = ",".join(map(str, current))
    dfa_transitions[current_label] = {}
    row = f"{{{current_label}}}".ljust(15)
    for symbol in symbol_set:
        target = move(current, symbol)
        target_label = ",".join(map(str, target))
        if target:
            target_label = f"{{{target_label}}}"
        else:
            target_label = "-"
        row += target_label.ljust(20)
        dfa_transitions[current_label][symbol] = target_label
        if target and target not in dfa_states:
            dfa_states.append(target)
            queue.append(target)
    print(row)
# Final DFA states list
print("\nDFA States:")
for state in dfa_states:
    print("{" + ",".join(map(str, state)) + "}")
```

`

SAMPLE INPUT & OUTPUT:

```
Enter number of NFA states: 3
Enter the NFA transi on table row by row.
Use 'e' for epsilon, 'a'/'b' for symbols,
'.' for none. Example for 3 states: e.a
..a ...
State 1: e.a
State 2: ..a
State 3: ...
Epsilon Closures:
 E(1) = {1} E(2) =
 {2}
 E(3) = {3}
 DFA
Transi on
Table:
State   a      b
-----------------------------------------------------------
{1}    {3}    {-}
{3}    {-}    {-}
 DFA States:
{1}
{3}
```

Result :

The NFA was constructed successfully using Thompson‟s method. All transitions and states were verified.

Exp No: 6	Implementation of Shift reduce parsing algorithm
Date:	

Aim:

To construct a Deterministic Finite Automaton (DFA) equivalent to a given Non-Deterministic Finite Automaton (NFA) using the Subset Construction Method.

Algorithm:

1) Start the program.
2) Read the input string and grammar rules in the form of production rules ($A \rightarrow \alpha$).
3) Initialize an empty stack to hold symbols and an input buffer with the given input string.
4) Repeat the following steps until the input string is completely parsed or an error occurs:
   a. Shift: Move (push) the next input symbol from the input buffer onto the stack.
   b. Reduce:
      a) After every shift, check whether the top of the stack matches the right-hand side (RHS) of any production rule.
      b) If a match is found, replace (reduce) the RHS on the stack with the left-hand side (LHS) non-terminal of that production.
5) Continue steps (a) and (b) until:
   a) The stack contains only the start symbol, and
   b) The input buffer is empty.
6) If the stack contains the start symbol and input is empty,
   → The string is successfully parsed
   (Accepted). Otherwise, report an
   error (Rejected).
7) Stop the program.

PROGRAM:

```
def shift_reduce_parser(input_str):
    stack = []
    ip = list(input_str)
    ip.append('$') # End marker
    pointer = 0

    print("\n\tShift Reduce Parser")
    print("\nGrammar:")
    print("E → E + E\nE → E * E\nE → E / E\nE → a | b")
    print("\nStack\t\tInput\t\tAction")

    def print_state(action):
        print(
            f"{''.join(stack):<16}"
            f"{''.join(ip[pointer:]):<16}"
            f"{action}"
        )

    def reduce_stack():
        nonlocal stack
        s = ''.join(stack)

        if s.endswith("E+E") or s.endswith("E*E") or s.endswith("E/E"):
            rule = s[-3:]
            stack = stack[:-3]
            stack.append('E')
            print_state(f"Reduce by E → {rule}")
            return True

        elif stack and stack[-1] in ['a', 'b']:
            stack[-1] = 'E'
            print_state(f"Reduce by E → {s[-1]}")
            return True

        return False

    print_state("--")

    while pointer < len(ip) - 1:
        stack.append(ip[pointer])
        pointer += 1
        print_state(f"Shift '{stack[-1]}'")

        while reduce_stack():
            pass
```

`

```
        while
        reduce_stack():
        pass

    if stack == ['E'] and ip[pointer] == '$':
        print_state("Accept")
    else:
        print_state("Reject")


# Input
input_expr = input(
    "Enter the input expression (only a, b, +, *, /): "
).strip()

shift_reduce_parser(input_expr)
```

SAMPLE INPUT & OUTPUT:

```
Enter the input EXPression (only a, b, +, *, /): if(a*b)
Shift Reduce Parser
Grammar:
E → E + E
E → E * E
E → E / E
E → a | b
Stack      Input    Action
           if(a*b)$ --
   i        f(a*b)$ Shift 'i'
   if       (a*b)$   Shift 'f'
   if(       a*b)$  Shift '('
   if(a      *b)$     Shift 'a'
   if(E      *b)$    Reduce by E → a
   if(E*     b)$     Shift '*'
   if(E*b    )$      Shift 'b'
   if(E*E    )$      Reduce by E → b
   if(E      )$      Reduce by E → E*E
   if(E)     $       Shift‟)‟
   if(E)     $       Reject
```

`

SAMPLE INPUT & OUTPUT:

```
Enter the input EXPression (only a, b, +, *, /): if(a*b)
Shift Reduce Parser
Grammar:
E → E + E
E → E * E
E → E / E
E → a | b
Stack      Input    Action
           if(a*b)$ --
   i        f(a*b)$ Shift 'i'
   if       (a*b)$   Shift 'f'
   if(       a*b)$  Shift '('
   if(a      *b)$    Shift 'a'
   if(E      *b)$    Reduce by E → a
   if(E*     b)$     Shift '*'
   if(E*b    )$      Shift 'b'
   if(E*E    )$      Reduce by E → b
   if(E      )$      Reduce by E → E*E
   if(E)     $       Shift“)“
   if(E)     $       Reject
```

RESULT :

The shift-reduce parser executed successfully. Input strings were parsed according to the grammar rules.

Exp No: 7	Implementation of Operator precedence parser
Date:	

AIM:

To implement an Operator Precedence Parser for a given grammar and input string in order to parse arithmetic or logical expressions using operator precedence relations.

ALGORITHM:

1. Start the program.
2. Read the input grammar and identify all operators and operands.
3. Construct the Operator Precedence Table showing the precedence relations between operators:
   < (less than → shift)
   = (equal → handle same precedence)
   > (greater than → reduce)
4. Initialize a stack with a bottom marker symbol (e.g., $).
5. Repeat the following steps until the input string is completely parsed:

a. Compare the precedence of the symbol on top of the stack with the next input symbol.

b. If precedence is < or =, Shift the next input symbol onto the stack.

c. If precedence is >, Reduce by replacing the handle (a recognized right-hand side pattern) on the stack with its corresponding non-terminal.

d. If no valid precedence relation exists, report an error.

6. If the stack contains only the start symbol and the input is fully read (just $ remains), then accept the input string; otherwise, reject it.
7. Stop the program.

PROGRAM:

```
# Operator Precedence Parser in Python 3.7

# Precedence table (converted from C version)
q = [
    ['>', '>', '<', '<', '<', '<', '>', '<', '>'],
    ['>', '>', '<', '<', '<', '<', '>', '<', '>'],
```

`

```
    ['>', '>', '>', '>', '<', '<', '>', '<', '>'],
    ['>', '>', '>', '>', '<', '<', '>', '<', '>'],
    ['>', '>', '<', '<', '<', '<', '>', '<', '>'],
    ['<', '<', '<', '<', '<', '<', '=', '<', 'E'],
    ['>', '>', '>', '>', '>', 'E', '>', 'E', '>'],
    ['>', '>', '>', '>', '>', 'E', '>', 'E', '>'],
    ['<', '<', '<', '<', '<', '<', 'E', '<', 'A']
]

stack = []
qs = []
p = 0


# Find index in precedence table
def find(a):
    if a == '+':
        return 0
    elif a == '-':
        return 1
    elif a == '*':
        return 2
    elif a == '/':
        return 3
    elif a == '^':
        return 4
    elif a == '(':
      return 5
    elif a == ')':
      return 6
    elif a.isalpha():
        return 7
    elif a == '$':
        return 8
    else:
        return -1


# Display shift action
def display(a):
    print(f"\nShift {a}")


# Display reduce action
def display1(a):
    if a.isalpha():
        print(f"\nReduce E -> {a}")
    elif a in ['+', '-', '*', '/', '^']:
        print(f"\nReduce E -> E{a}E")
    elif a == ')':
        print("\nReduce E -> (E)")
`
```

```
        # Relation check based on precedence
table def rel(a, b, d):
    if a.isalpha():
        a = 'a'

    if b.isalpha():
        b = 'a'

    return q[find(a)][find(b)] == d


# Main function
def operator_precedence_parser():
    global p

    # Operator Precedence Parser
    expr = input(
        "\nEnter the Arithmetic Expression ending with $: "
    ).strip()

    stack.append('$')

    while True:
        if expr[p] == '$' and stack[-1] == '$':
            print("\n\nAccepted")
            break
        elif rel(stack[-1], expr[p], '<') or rel(stack[-1], expr[p], '='):
            display(expr[p])
            stack.append(expr[p])
            p += 1
        elif rel(stack[-1], expr[p], '>'):
            while True:
                qs.append(stack.pop())
                display1(qs[-1])
                if rel(stack[-1], qs[-1], '<'):
                    break
# Run the parser
operator_precedence_parser()
```

`

SAMPLE INPUT/OUTPUT:

Enter the Arithmetic Expression ending with $:a*b$

Shift a

Reduce E -> a

Shift *

Shift b

Reduce E -> b

Reduce E -> E*E

RESULT :

The operator precedence parser worked correctly. Expressions were parsed based on defined precedence relations

`

Exp No: 8	Implementation of recursive descent parser
Date:	

AIM:

To implement a Recursive Descent Parser for a given grammar and input string in order to check whether the string belongs to the language defined by the grammar.

ALGORITHM:

1) Start the program.
2) Read the input string and the grammar rules.
3) Eliminate left recursion and perform left factoring (if required) so that the grammar is suitable for recursive descent parsing.
   a) Write one procedure (function) for each non-terminal in the grammar:
   b) Each procedure implements its corresponding production rules.
   c) Terminals are matched directly with the input symbols.
4) For each non-terminal on the right-hand side, call its corresponding procedure.
5) Maintain a pointer to the current input symbol.
   a) When a match occurs, advance the pointer.
   b) If a mismatch occurs, report a syntax error and terminate parsing.
6) Call the start symbol‟s procedure from the main program.
   a) If the input string is fully consumed and no error occurs, print “String accepted.”
   b) Otherwise, print “String rejected.”
7) Stop the program.

PROGRAM:

```
# Recursive Descent Parser in Python

ip_sym = ""
ip_ptr = 0
```

`

```
def advance():
    global ip_ptr
    ip_ptr += 1

def e():
    print("\n\t\tE -> T E'")
    t()
    e_prime()

def e_prime():
    global ip_sym, ip_ptr
    if ip_ptr < len(ip_sym) and ip_sym[ip_ptr] == '+':
        print("\n\t\tE' -> + T E'")
        advance()
        t()
        e_prime()
    else:
        print("\n\t\tE' -> ε")
def t():
    print("\n\t\tT -> F T'")
    f()
    t_prime()
def t_prime():
    global ip_sym, ip_ptr
    if ip_ptr < len(ip_sym) and ip_sym[ip_ptr] == '*':
        print("\n\t\tT' -> * F T'")
        advance()
        f()
        t_prime()
    else:
        print("\n\t\tT' -> ε")
def f():
    global ip_sym, ip_ptr
```

`

```
    if ip_ptr < len(ip_sym) and (
        ip_sym[ip_ptr] == 'i' or ip_sym[ip_ptr] == 'j'
    ):
        print("\n\t\tF -> id")
        advance()

    elif ip_ptr < len(ip_sym) and ip_sym[ip_ptr] == '(':
        advance()
        e()

        if ip_ptr < len(ip_sym) and ip_sym[ip_ptr] == ')':
            advance()
            print("\n\t\tF -> (E)")
        else:
            print("\n\t\tSyntax Error: missing ')'")
            exit(1)

    else:
        print("\n\t\tSyntax Error: unexpected symbol")
        exit(1)

# Main function
def main():
    global ip_sym

    print("\n\t\tGRAMMAR WITHOUT LEFT RECURSION")
    print("\n\t\tE -> T E'")
    print("\t\tE' -> + T E' | ε")
    print("\t\tT -> F T'")
    print("\t\tT' -> * F T' | ε")
    print("\t\tF -> (E) | id")

    ip_sym = input("\n\t\tEnter the input string: ").strip()

    print("\n\t\tSequence of Production Rules:")
    e()
```

`

```
        if ip_ptr == len(ip_sym):
        print("\n\t\tInput accepted!")
    else:
        print("\n\t\tSyntax Error: extra symbols")


if __name__ == "__main__":
    main()
```

`

SAMPLE INPUT/OUTPUT:

GRAMMAR WITHOUT LEFT RECURSION

E -> T E'
E' -> + T E' | ε
T -> F T'
T' -> * F T' | ε
F -> (E) | id

Enter the input string: i+j

Sequence of Production Rules:

E -> T E'

T -> F T'

F -> id

T' -> ε

E' -> + T E'

T -> F T'

F -> id

T' -> ε

E' -> ε

Input accepted!

RESULT :

The recursive descent parser was implemented successfully. Input strings were parsed as per the LL(1) grammar.

`

Exp No: 9	Implementation of code optimization techniques
Date:	

AIM:

To implement various code optimization techniques to improve the efficiency of intermediate code by reducing redundant operations and enhancing execution speed.

ALGORITHM:

1) Start the program.
2) Read the intermediate code or three-address code (TAC) generated by the compiler.
3) Identify optimization opportunities in the code such as:
   a) Constant folding: Evaluate constant expressions at compile time.
   b) Constant propagation: Replace variables having constant values with the constants themselves.
   c) Dead code elimination: Remove statements whose results are never used.
   d) Common subexpression elimination: Identify and eliminate repeated expressions computed multiple times.
   e) Strength reduction: Replace costly operations with equivalent cheaper ones (e.g., replace multiplication with addition).
4) Traverse the intermediate code statement by statement and apply the above optimizations where applicable.
5) Generate the optimized intermediate code by replacing or removing redundant instructions.
6) Display the optimized code as the output.
7) Stop the program.

PROGRAM:

```
class Operation:
    def __init__(self, left, right):
        self.l = left
        self.r = right

def main():
    n = int(input("Enter the number of values: "))
    operations = []

    # Input intermediate code
    for i in range(n):
        left = input(f"Left operand {i+1}: ").strip()
        right = input(f"Right expression {i+1}: ").strip()
        operations.append(Operation(left, right))
```

`

```
        print("\nIntermediate
    Code:") for op in operations:
        print(f"{op.l} = {op.r}")
    # Dead Code Elimination
    pr = []
    for i in range(n - 1):
        temp = operations[i].l
        for j in range(n):
            if temp in operations[j].r:
                pr.append(Operation(operations[i].l, operations[i].r))
                break
    # Add last operation
    pr.append(Operation(operations[n - 1].l, operations[n - 1].r))
    print("\nAfter Dead Code Elimination:")
    for op in pr:
        print(f"{op.l} = {op.r}")
    # Common Subexpression Elimination
    for m in range(len(pr)):
        tem = pr[m].r
        for j in range(m + 1, len(pr)):
            if pr[j].r in tem:
                t = pr[j].l
                pr[j].l = pr[m].l
                # Replace occurrences
                for i in range(len(pr)):
                    if t in pr[i].r:
                        pr[i].r = pr[i].r.replace(t, pr[m].l)
    print("\nEliminate Common Expression:")
    for op in pr:
        print(f"{op.l} = {op.r}")
    # Remove duplicates
    optimized = []
    for i in range(len(pr)):
        duplicate = False
        for j in range(i + 1, len(pr)):
            if pr[i].r == pr[j].r and pr[i].l == pr[j].l:
                duplicate = True
                break
        if not duplicate:
            optimized.append(pr[i])
    print("\nOptimized Code:")
if __name___== "__main__":
    main()
```

`

SAMPLE INPUT/OUTPUT:

Enter the number of values: 3

Left operand 1: a

Right expression 1: 3

Left operand 2: b

Right expression 2: a+b

Left operand 3: e

Right expression 3: a+b

Intermediate Code:

a = 3

b = a+b

e = a+b

After Dead Code Elimination:

a = 3

b = a+b

e = a+b

Eliminate Common Expression:

a = 3

b = a+b

b = a+b

Optimized Code:

a = 3

b = a+b

RESULT :

The optimization program executed successfully. Redundant computations were removed and expressions simplified.

`

Exp No: 10	Implementation of code generator
Date:	

AIM:

To implement a simple code generator that converts intermediate code (three-address code) into target machine code or assembly code.

ALGORITHM:

1) Start the program.
2) Read the intermediate code (three-address code) generated by the intermediate code generator phase of the compiler.
3) Initialize registers and memory locations required for storing operands and results.
   a) For each three-address statement, perform the following steps:
      a. Identify the operator and operands.
      b. Generate target code instructions for:
   b) Loading operands into registers.
   c) Performing the specified operation (e.g., ADD, SUB, MUL, DIV).
4) Storing the result back into a register or memory location.
5) Apply simple optimizations to minimize register usage (e.g., reuse registers for temporary variables).
6) Output the final target machine/assembly code corresponding to the input intermediate code.
7) Stop the program.

PROGRAM:

```
class Register:
    def __init__(self):
        self.var = ""
        self.alive = False


# Initialize 10 registers
preg = [Register() for _ in range(10)]


def substring(exp, start, end):
    return exp[start:end]
```

`

```
def get_register(var):
    for i in range(10):
        if not preg[i].alive:
            preg[i].var = var
            return i
    return -1


def get_var(exp):
    var = ""
    for ch in exp:
        if ch.isalpha():
            var += ch
        else:
            break
    return var


def main():
    basic = []
    print("\nEnter the Three Address Code (type 'exit' to finish):")

    while True:
        line = input().strip()
        if line.lower() == "exit":
            break
        basic.append(line)

    print("\nThe Equivalent Assembly Code is:")
    vc = 0

    for j in range(len(basic)):
        exp = basic[j]

        # Get first variable
        var1 = get_var(exp)
        vc += 1
        fstr = var1
        exp = substring(exp, len(var1) + 1, len(exp))

        # Get second variable
        var2 = get_var(exp)
        vc += 1
        reg = get_register(var2)

        if not preg[reg].alive:
            print(f"\nMov R{reg}, {var2}")
            preg[reg].alive = True
`
```

```
        # Get operator
        op = exp[len(var2)]
        exp = substring(exp, len(var2) + 1, len(exp))

        # Get third variable
        var3 = get_var(exp)
        vc += 1

        # Print operation
        if op == '+':
            print("Add", end='')
        elif op == '-':
            print("Sub", end='')
        elif op == '*':
            print("Mul", end='')
        elif op == '/':
            print("Div", end='')

        # Check if var3 is in any register
        flag = True
        for k in range(reg + 1):
            if preg[k].var == var3:
                print(f" R{k}, R{reg}")
                preg[k].alive = False
                flag = False
                break

        if flag:
            print(f" {var3}, R{reg}")

        print(f"\nMov {fstr}, R{reg}")
        preg[reg].var = fstr


if __name__ == "__main__":
    main()
```

`

SAMPLE INPUT/OUTPUT:

Enter the Three Address Code (type 'exit' to finish):

a=b+c

c=a*c

exit

The Equivalent Assembly Code is:

```
Mov R0, b
Add c, R0

Mov a, R0

Mov R1, a
Mul c, R1

Mov c, R1
```

RESULT :

The code generator worked successfully. Correct intermediate and target code were produced for sample inputs.

`

Exp No: 11	Implementation of symbol table
Date:	

AIM:

To implement a Symbol Table that stores and manages information about identifiers such as variables, functions, and constants during compilation.

ALGORITHM:

1) Start the program.
2) Initialize the symbol table as an empty data structure (e.g., array, linked list, or hash table).
3) Read the source program or list of tokens generated by the lexical analyzer.
4) For each token encountered in the source code:

   a. If the token is an identifier, search the symbol table to check if it already exists.

   b. If it does not exist, insert a new entry with the following information:

   a) Name of the identifier
   b) Type (e.g., int, float, char)
   c) Scope (global/local)
   d) Memory location or address
   e) Value (if applicable)

   c. If it already exists, update the entry (for re-declaration, new value, or scope change).
5) Allow operations such as:
   a) Insertion of a new symbol
   b) Deletion of a symbol
   c) Modification of symbol attributes
   d) Search for an existing symbol
6) Display the contents of the symbol table in tabular form (showing all identifiers and their attributes).
7) Stop the program.

`

PROGRAM:

```
class Symbol:
    def __init__(self, label, addr):
        self.label = label
        self.addr = addr
        self.next = None


class  SymbolTable:
    def __init__(self):
        self.first = None
        self.last  =  None
        self.size = 0

    def insert(self, label, addr):
        if self.search(label):
            print("The label already exists. Duplicate can't be inserted.")
            return

        new_node = Symbol(label, addr)

        if self.size == 0:
            self.first = self.last = new_node
        else:
            self.last.next = new_node
            self.last = new_node

        self.size += 1

    def display(self):
        print("\nLABEL\tADDRESS")
        temp = self.first

        while temp:
            print(f"{temp.label}\t{temp.addr}")
            temp = temp.next

    def search(self, label):
        temp = self.first

        while temp:
            if temp.label == label:
                return True
            temp = temp.next

        return False

    def modify(self, label, new_label=None, new_addr=None):
        temp = self.first
```

`

```
            while temp:
             if temp.label == label:
                if new_label:
                    temp.label = new_label
                if new_addr is not None:
                    temp.addr = new_addr
                return
             temp = temp.next
        print("No such label.")
        return False
    def delete(self, label):
        prev = None
        temp = self.first

        while temp:
            if temp.label == label:
                if prev:
                    prev.next = temp.next
                else:
                    self.first = temp.next

                if temp == self.last:
                    self.last = prev

                self.size -= 1
                return True

            prev = temp
            temp = temp.next

        print("Label not found.")
        return False


# Driver code
def main():
    table = SymbolTable()

    while True:
        print("\nSYMBOL TABLE IMPLEMENTATION")
        print("1. INSERT")
        print("2. DISPLAY")
```

`

SAMPLE INPUT/OUTPUT:

```
SYMBOL TABLE IMPLEMENTATION
1. INSERT
2. DISPLAY
3. DELETE
4. SEARCH
5. MODIFY
6. EXIT
Enter your option: 1
Enter the label: a
Enter the address: 5
LABEL  ADDRESS
a      5
Enter your option: 2

LABEL   ADDRESS
a       5
Enter your option: 3
Enter the label to delete: a

LABEL   ADDRESS
Enter your option: 4
Enter the label to search: a
Not Found
Enter your option: 5
Enter the label to modify: a
Enter new label (or press Enter to skip): a
Enter new address (or press Enter to skip): 5
No such label.

Enter your option: 6
Exits from the program
```

RESULT :

The symbol table was implemented successfully. Entries were inserted and retrieved accurately with scope handling.

`